L04. Combinational Logic (3)

Lecture 04

4.1 2단 논리 최소화

> **정준형으로 쓴 뒤 인접한 항을 묶으면 더 간단한 2단 논리. SoP 는 출력 1인 minterm 을 묶어 곱항을, PoS 는 출력 0인 maxterm 을 묶어 합항을 줄임.**

logic minimization: 같은 함수를 더 적은 비용(게이트 수, 실리콘 면적)으로 구현.

- 2단 논리 최소화 목표: SoP 는 ① 곱항 수 최소 ② 곱항 수가 같으면 전체 리터럴 수 최소. PoS 는 ① 합항 수 최소 ② 합항 수가 같으면 전체 리터럴 수 최소. 두 경우 모두 원래 논리함수를 유지하면서 2단 구현의 비용을 줄이는 것이 목표다.
- 대수적 변형만으로는 최소식을 찾았는지 보장할 수 없음. 변형 순서에 따라 더 간단한 식을 놓칠 수 있음 (local minimum).
- 체계적인 방법: 불리언 큐브, 카르노 맵, Quine-McCluskey 방법.

4.2 불리언 큐브 (Boolean cube)

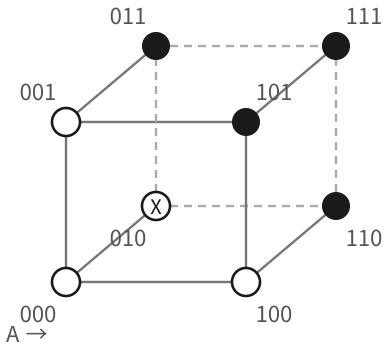
- 부울식의 시각적 표현. 축 하나가 변수 하나. positional notation로 A′BC′ → 010. 노드 하나 = 입력 조합 하나. 각 입력 조합에는 minterm 하나, maxterm 하나씩 대응.
- 1-cube(변수 1개) 선분, 2-cube(변수 2개) 정사각형, 3-cube(변수 3개) 정육면체.

진리표를 큐브에 옮기기

- ON-set = 검은 노드, OFF-set = 흰 노드, DC-set(don’t care) = X 노드.

예

A	B	C	F
0	0	0	0
0	0	1	0
0	1	0	X
0	1	1	1
1	0	0	0
1	0	1	1
1	1	0	1
1	1	1	1



ON-set 은 011, 101, 110, 111. 010 은 X.

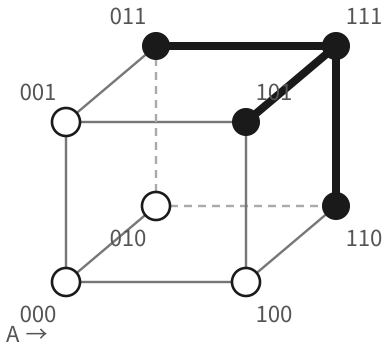
인접한 minterm 합치기

> uniting theorem XY + XY′ = X 를 sub-cube으로 그림. 한 변수만 다른 인접한 ON-set minterm 을 묶으면 그 변수가 사라짐. 큰 원 하나 = 이렇게 합친 곱항 하나.

- F = AB′ + AB = A (11, 10 에서 B 소거).
- G = A′B′ + AB′ = B′.
- H = AB + AB′ + A′B′ → A, B′ 두 묶음. H = A + B′.

예 · Full adder 의 C_{out}

A	B	C_{in}	C_{out}
0	0	0	0
0	0	1	0
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	1
1	1	0	1
1	1	1	1



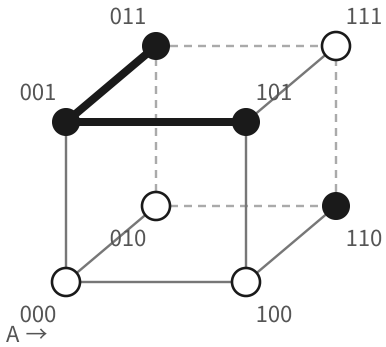
굵은 선이 각각 하나의 1-sub-cube.

- 011 과 111 → −11. $A'BC_{in} + ABC_{in} = (A'+A)BC_{in} = BC_{in}$.
- 110 과 111 → 11−. $AB(C_{in}'+C_{in}) = AB$.
- 101 과 111 → 1−1. $A(B+B')C_{in} = AC_{in}$.

$C_{out} = BC_{in} + AB + AC_{in}$

예 · 연습

$Z = A'B'C + A'BC + AB'C + ABC' = \Sigma m(1,3,5,6)$



001 은 011(A′C), 101(B′C)과 각각 인접한다. 110 은 인접한 ON 노드가 없다.

$Z = A'C + B'C + ABC'$ (001 은 두 묶음에 겹쳐 씀)

고차원 인접성: sub-cube 의 차원과 리터럴 수

sub-cube	노드 수	3-cube 에서의 곱항	예
0-cube	1	리터럴 3개	A′BC
1-cube	2	리터럴 2개	BC (011 & 111 에서 A 가 사라짐)
2-cube	4	리터럴 1개	A (100, 101, 111, 110 에서 B 와 C 가 사라짐)
3-cube	8	리터럴 0개	1

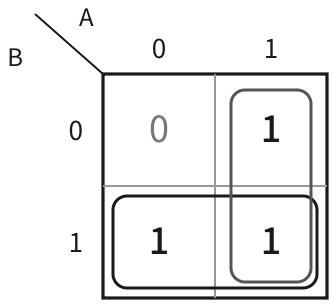
k-cube 은 2^k 개 노드를 포함. 묶음 안에서 값이 바뀌는 k 개 변수가 사라지므로 곱항에는 $n - k$ 개 리터럴이 남음.

4.3 카르노 맵 (K-map)

- K-map = 큐브처럼 진리표를 다르게 배치한 것. 인접한 칸 = 한 변수만 다른 조합. 그 칸들을 묶어 uniting theorem 적용.
- 큐브는 3차원을 넘으면 그리기 어려움. 다차원 공간을 2차원에 투영한 게 K-map.

- **Gray code**: 다음 인덱스로 갈 때 한 비트만 바뀜. 이웃한 두 칸의 거리는 1.
- 한쪽 가장자리 칸은 반대쪽 가장자리 칸과 인접.

2변수



F = A′B + AB′ + AB. 두 묶음: B (아랫줄), A (오른쪽 열). F = A + B.

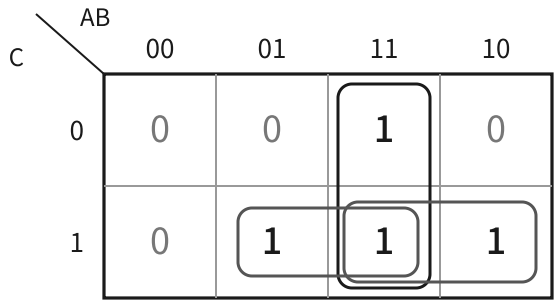
- 칸 하나 = 진리표 행 하나. 칸 안의 값이 출력. 보통 1(과 0, X)만 적음.
- 인접한 두 칸을 묶으면 값이 다른 변수가 사라짐. 한 칸은 리터럴 2개(AB′), 두 칸 묶음은 리터럴 1개(A). 확인은 AB′ + AB = A, 또는 그 변수에 0과 1이 다 들어 있는지 보면 됨.
- 묶음 수 = 곱항 수. 묶음이 클수록 리터럴이 적음. 묶음은 1, 2, 4, 8 … 칸의 직사각형만 가능하고 겹쳐도 됨. 최소 SoP = 1인 칸을 전부 덮는 묶음 조합 중 곱항 수, 리터럴 수가 최소인 것.

3변수

- minterm 위치와 인접한 칸 확인. 011(A′BC)의 이웃은 001(A′B′C), 010(A′BC′), 111(ABC).
- SoP 는 출력 1인 칸(minterm)을, PoS 는 출력 0인 칸(maxterm)을 K-map 에 옮겨 묶음. 예: F(a,b,c) = $m_1 + m_3 + m_5 = M_0M_2M_4M_6M_7$.
- 묶음을 식으로 읽는 법: SoP 는 묶음 안에서 값이 일정한 변수 중 1인 변수는 그대로, 0인 변수는 보수를 취해 곱항을 만든다. PoS 는 반대로 0인 변수는 그대로, 1인 변수는 보수를 취해 합항을 만든다.
- PoS 최소화 예: F(A,B,C) = ΠM(0,2,6,7). 0인 칸 m_0, m_2 를 묶으면 A + C, 0인 칸 m_6, m_7 을 묶으면 A′ + B′. 따라서 최소 PoS 는 F = (A + C)(A′ + B′) 이다.

	AB=00	01	11	10
C=0	m_0	m_2	m_6	m_4
C=1	m_1	m_3	m_7	m_5

예 · F = AB + AC + BC

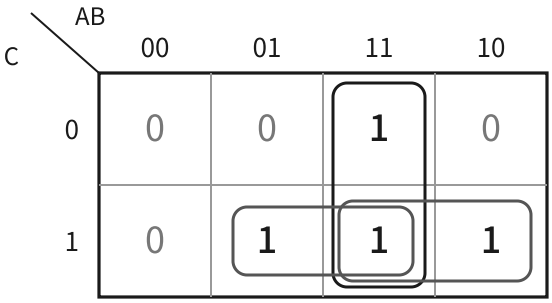


F = Σm(3,5,6,7). 세 묶음이 AB, AC, BC.

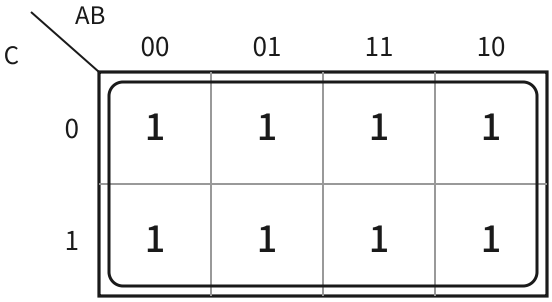
A	B	C	F
0	0	0	0
0	0	1	0
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	1
1	1	0	1
1	1	1	1

F = AB + AC + BC

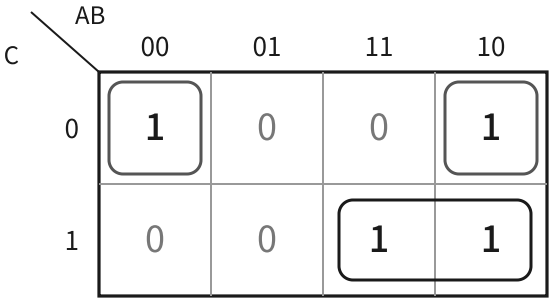
예



$C_{out} = BC_{in} + AB + AC_{in}$



모든 칸이 1이면 F = 1 (묶으면 모든 변수가 사라진다).



F = AC + B′C′. 가장자리의 두 1은 wrap-around 로 이어진다.

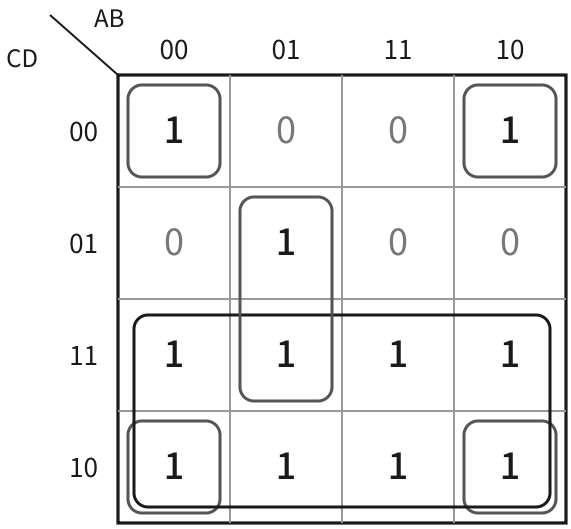
4변수

- 3변수 맵과 같은 규칙. 첫 열과 마지막 열, 첫 행과 마지막 행이 서로 인접.

	AB=00	**01**	**11**	**10**
CD=00	m_0	m_4	m_{12}	m_8
01	m_1	m_5	m_{13}	m_9
11	m_3	m_7	m_{15}	m_{11}
10	m_2	m_6	m_{14}	m_{10}

- 묶으면 2개 → 리터럴 3개, 4개 → 2개, 8개 → 1개 남음. four corners도 한 묶음.

예 · F(A,B,C,D) = Σm(0,2,3,5,6,7,8,10,11,14,15)



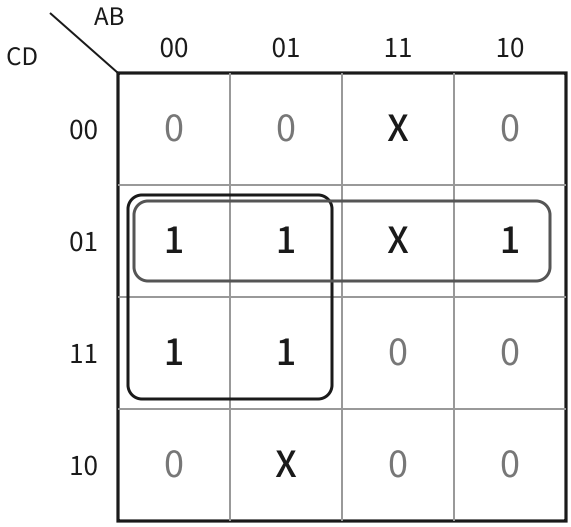
네 모서리의 4칸을 묶으면 B′D′, 아래 두 행의 8칸을 묶으면 C 가 된다. 아직 덮이지 않은 m5 는 인접한 m7 과 2칸으로 묶어 A′BD 를 얻는다. 이때 m7 은 C 묶음과 중복 사용한다.

F = A′BD + B′D′ + C

Don’t care 다루기

- don’t care minterm 은 X 로 표시. 최소 SoP 는 1을 전부 덮어야 하고, X 는 묶음이 커져 식이 간단해질 때만 1 로 사용. 도움이 안 되는 X 는 0 으로 두고 묶지 않음.

예 · F(A,B,C,D) = Σm(1,3,5,7,9) + Σd(6,12,13)



A′D 는 m1,3,5,7. C′D 는 m1,5,9,13 (13 은 X 를 1로 씀). 6 과 12 는 쓰지 않는다.

F = A′D + C′D